

针对您的需求，我将提供一套完整的解决方案，实现基于 **EPSG:4326** 坐标系的矢量切片服务及前端加载。传统 MVT 方案默认采用 EPSG:3857，而本方案核心在于**统一前后端的瓦片网格定义**，使瓦片在经纬度投影下无畸变显示。

1. 整体方案设计

坐标系选择与网格定义

- 使用 EPSG:4326 (等矩形投影)**：经度范围 `[-180, 180]`，纬度范围 `[-90, 90]`。
- 保持瓦片为正方形地理范围**：在缩放级别 `z` 下，整个世界被划分为 2^z 行（纬度方向）和 $2^{(z+1)}$ 列（经度方向），每个瓦片覆盖相同大小的经纬度跨度（`tileDeg = 180 / 2^z` 度）。
- 瓦片原点**：左上角 `(-180, 90)`，符合 XYZ 瓦片编号规则（Y 轴向下）。

前后端协同

- 前端 (OpenLayers)**：自定义 `TileGrid`，明确 `resolutions` 和 `origin`，使用 `VectorTileSource` 加载 MVT。
- 后端 (Spring Boot + PostGIS)**：根据 `z/x/y` 计算瓦片边界，利用 `ST_AsMVT` 生成 MVT 二进制。

2. 后端实现：Spring Boot + PostGIS

2.1 数据库准备

确保 PostGIS 扩展已启用：

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

假设测试表 `public.places` 存储了全球的 POI 数据 (SRID=4326)：

```
CREATE TABLE places (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  geom GEOMETRY(Point, 4326)  
);  
CREATE INDEX idx_places_geom ON places USING GIST(geom);
```

2.2 瓦片边界计算逻辑

瓦片编号规则：z 级时有 $rows = 2^z$, $cols = 2^{(z+1)}$ 。

瓦片行列号 (x, y) 对应边界：

- $tileDeg = 180.0 / (1 \ll z)$ (纬度高度)
- $minLon = -180 + x * tileDeg$
- $maxLat = 90 - y * tileDeg$
- $minLat = maxLat - tileDeg$

2.3 矢量切片 SQL

利用 `ST_TileEnvelope` 函数可以简化，但为了更精确控制（正方形瓦片），这里直接构造边界几何：

```
WITH
  -- 计算瓦片边界
  tile_bounds AS (
    SELECT
      ST_MakeEnvelope(
        -180 + (${x}::numeric * (180.0 / POWER(2, ${z}))), -- minLon
        90 - (${y}::numeric + 1) * (180.0 / POWER(2, ${z})), -- minLat
        -180 + ((${x}::numeric + 1) * (180.0 / POWER(2, ${z}))), -- maxLon
        90 - ${y}::numeric * (180.0 / POWER(2, ${z})), -- maxLat
        4326
      ) AS geom
  ),
  -- 裁剪与坐标变换
  clipped AS (
    SELECT
      id,
      name,
      ST_AsMVTGeom(
        p.geom,
        tb.geom,
        4096,
        0,
        true
      ) AS mvt_geom
    FROM places p, tile_bounds tb
    WHERE p.geom && tb.geom -- 空间索引加速
  )
SELECT ST_AsMVT(clipped.*, 'places', 4096, 'mvt_geom') AS mvt
FROM clipped;
```

说明：

- `tile_bounds` 根据当前 `z/x/y` 计算正方形边界（经度跨度 = 纬度跨度）。
- `ST_AsMVTGeom` 将几何裁剪到边界并转换为 MVT 内部像素坐标系（范围 0~4096）。
- `ST_AsMVT` 聚合所有行生成一个矢量瓦片。

2.4 Spring Boot 控制器

```
@RestController
@RequestMapping("/tiles")
public class TileController {

    @Autowired
    private DataSource dataSource;

    @GetMapping("/{z}/{x}/{y}.mvt")
    public ResponseEntity<byte[]> getTile(
        @PathVariable int z,
        @PathVariable int x,
        @PathVariable int y) {

        // 简单校验
        if (z < 0 || x < 0 || y < 0) {
            return ResponseEntity.badRequest().build();
        }

        String sql = """
            WITH
              tile_bounds AS (
                SELECT ST_MakeEnvelope(
                  -180 + :x * (180.0 / POWER(2, :z)),
                  90 - (:y + 1) * (180.0 / POWER(2, :z)),
                  -180 + (:x + 1) * (180.0 / POWER(2, :z)),
                  90 - :y * (180.0 / POWER(2, :z)),
                  4326
                ) AS geom
              ),
              clipped AS (
                SELECT
                  id,
                  name,
                  ST_AsMVTGeom(p.geom, tb.geom, 4096, 0, true) AS mvt_geom
                FROM places p, tile_bounds tb
                WHERE p.geom && tb.geom
              )
            SELECT ST_AsMVT(clipped.*, 'places', 4096, 'mvt_geom') FROM clipped
            """;

        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, x);
            ps.setInt(2, z);
            ps.setInt(3, y);
            ps.setInt(4, z);    // POWER(2, :z) 会复用
            // 注意: SQL 中使用了两种占位符, 为简化这里用命名参数可使用
            NamedParameterJdbcTemplate
                // 真实代码建议用 NamedParameterJdbcTemplate
        } catch (Exception e) { ... }
```

```
        // 返回 mvt 二进制, Content-Type: application/vnd.mapbox-vector-tile
    }
}
```

依赖配置 (使用 Spring Boot + JdbcTemplate) :

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/gis_db
    username: postgres
    password: postgres
```

3. 前端实现: OpenLayers 加载 EPSG:4326 矢量切片

3.1 安装依赖

```
npm install ol
```

引入模块:

```
import Map from 'ol/Map.js';
import View from 'ol/View.js';
import TileLayer from 'ol/layer/Tile.js';
import OSM from 'ol/source/OSM.js';
import VectorTileLayer from 'ol/layer/VectorTile.js';
import VectorTileSource from 'ol/source/VectorTile.js';
import MVT from 'ol/format/MVT.js';
import TileGrid from 'ol/tilegrid/TileGrid.js';
import { getTopLeft } from 'ol/extent.js';
```

3.2 自定义 TileGrid

```
const tileSize = 256;
const maxZoom = 18;
const extent = [-180, -90, 180, 90]; // 全球范围
const origin = getTopLeft(extent); // [-180, 90]

// 分辨率数组: 每个级别瓦片的高度度数 / 256
const resolutions = [];
for (let z = 0; z <= maxZoom; z++) {
  resolutions.push(180 / (tileSize * Math.pow(2, z)));
}
```

```
const tileGrid = new TileGrid({
  extent: extent,
  origin: origin,
  resolutions: resolutions,
  tileSize: [tileSize, tileSize],
});
```

3.3 创建地图与矢量切片图层

// 底图：标准 OSM (EPSG:3857) 可替换为 EPSG:4326 的栅格瓦片，这里仅作示意

```
const baseLayer = new TileLayer({
  source: new OSM(),
});

const vectorLayer = new VectorTileLayer({
  source: new VectorTileSource({
    tileGrid: tileGrid,
    projection: 'EPSG:4326',
    format: new MVT(),
    url: 'http://localhost:8080/tiles/{z}/{x}/{y}.mvt',
    // 可设置 attributions 等
  }),
  style: new Style({
    // 简单样式
    fill: new Fill({ color: 'rgba(255,0,0,0.2)' }),
    stroke: new Stroke({ color: '#ff0000', width: 2 }),
  }),
});

const map = new Map({
  target: 'map',
  layers: [baseLayer, vectorLayer],
  view: new View({
    projection: 'EPSG:4326', // 使用经纬度投影
    center: [0, 0],
    zoom: 2,
    maxZoom: 18,
  }),
});
```

3.4 关键解释

- 因为 `tileGrid` 的 `origin` 为左上角，且 Y 轴向下（`resolutions` 为正值时，`getTopLeft` 返回的 `y` 最大），因此 OpenLayers 内部会生成符合 XYZ 规范的瓦片坐标（左上 Y=0，向下递增）。这与我们后端计算的 Y 方向一致（Y=0 对应纬度 90，Y=1 对应纬度 90-tileDeg）。

- `VectorTileSource` 的 `projection` 设为 `'EPSG:4326'`，它会负责将瓦片内像素坐标逆算为经纬度。
- 如果地图容器并非 2:1 宽高比，视图会自动根据 `extent` 适配，保证不变形。

4. 验证前端投影与样式

可以在样式函数中测试点击事件或高亮，一切坐标返回均为经纬度（EPSG:4326）：

```
map.on('click', (evt) => {  
  console.log(evt.coordinate); // [lon, lat]  
});
```

5. 性能与扩展建议

- **后端**：对 `places` 表建立 GIST 索引，使用 `&&` 操作符快速过滤。
- **缓存**：可以对相同 `z/x/y` 的 MVT 添加 HTTP 缓存头（`Cache-Control`），或使用 Redis 缓存。
- **数据量**：当数据量大时，考虑在 `ST_AsMVT` 中限制返回行数（例如 `WHERE p.geom && tb.geom LIMIT ...`），或使用材料化视图预聚合。

6. 完整工程结构

```
backend/  
  src/main/java/com/example/tiles/  
    TileController.java  
  src/main/resources/application.yml  
  
frontend/  
  src/  
    main.js    (地图初始化代码)  
  index.html
```

通过上述方案，您即可实现完全基于 **EPSG:4326** 的矢量切片，前后端坐标系一致，且瓦片网格完美适配。如有任何细节问题，欢迎继续探讨。